Multi-Agent Bug Analysis and Test Case Generation

By-
Kavish Nag - 24070126085
Kunal Jhindal - 24070126098

Github Repo: https://github.com/kjuhi-18/bugforge

Problem Statement and Domain Justification

Problem statement

Software developers spend considerable time analyzing bug reports, identifying the root cause of errors, reproducing bugs, and writing test cases to prevent similar issues in the future. These activities are often performed manually and across separate tools, making the debugging and testing process time-consuming and error-prone.

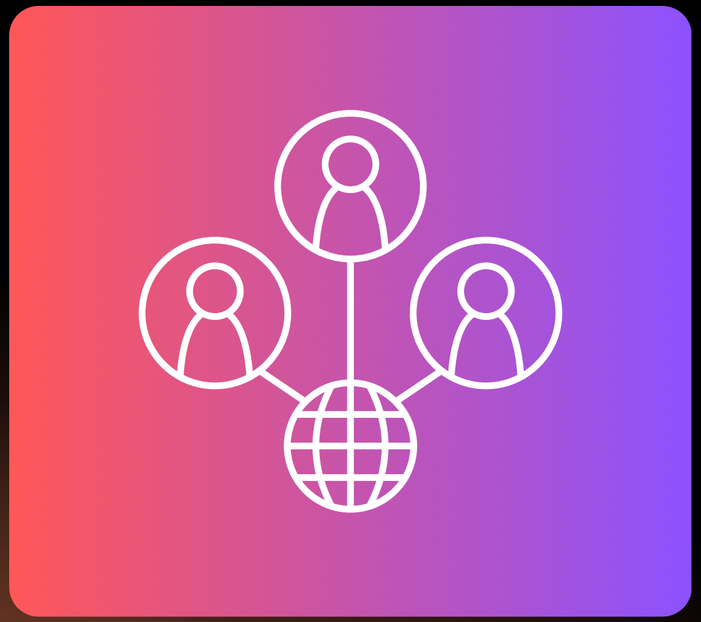
Existing challenges

- Bug reports may be incomplete, ambiguous, or difficult to reproduce.
- Developers must manually inspect code and trace possible causes.
- Test cases may overlook edge cases and regression scenarios.
- Debugging and testing activities are fragmented across the development workflow.

Proposed solution

Develop a multi-agent AI system that analyzes a bug report and associated code, identifies probable root causes, generates relevant test cases, executes supported tests, and validates the findings before presenting a final report.

Proposed Solution and Objectives



Automated bug analysis

Understand the bug report, inspect the relevant code, and identify likely causes with supporting evidence.

Automated test generation

Generate normal, boundary, invalid-input, and regression test cases based on the suspected bug.

Collaborative agent workflow

Assign specialized tasks to planning, analysis, test generation, execution, and validation agents.

Output validation

Check test completeness, execute permitted tests, identify failures, and produce a consolidated report.

Multi-Agent Architecture

1. Planner Agent

Breaks the request into tasks and determines their execution order.

2. Bug Analysis Agent

Examines the report and code to identify probable causes.

3. Test Generation Agent

Creates test cases from the bug analysis and expected behavior.

4. Test Execution Agent

Runs supported tests in a controlled environment.

5. Critic/Validation Agent

Checks the findings, test coverage, consistency, and execution evidence.

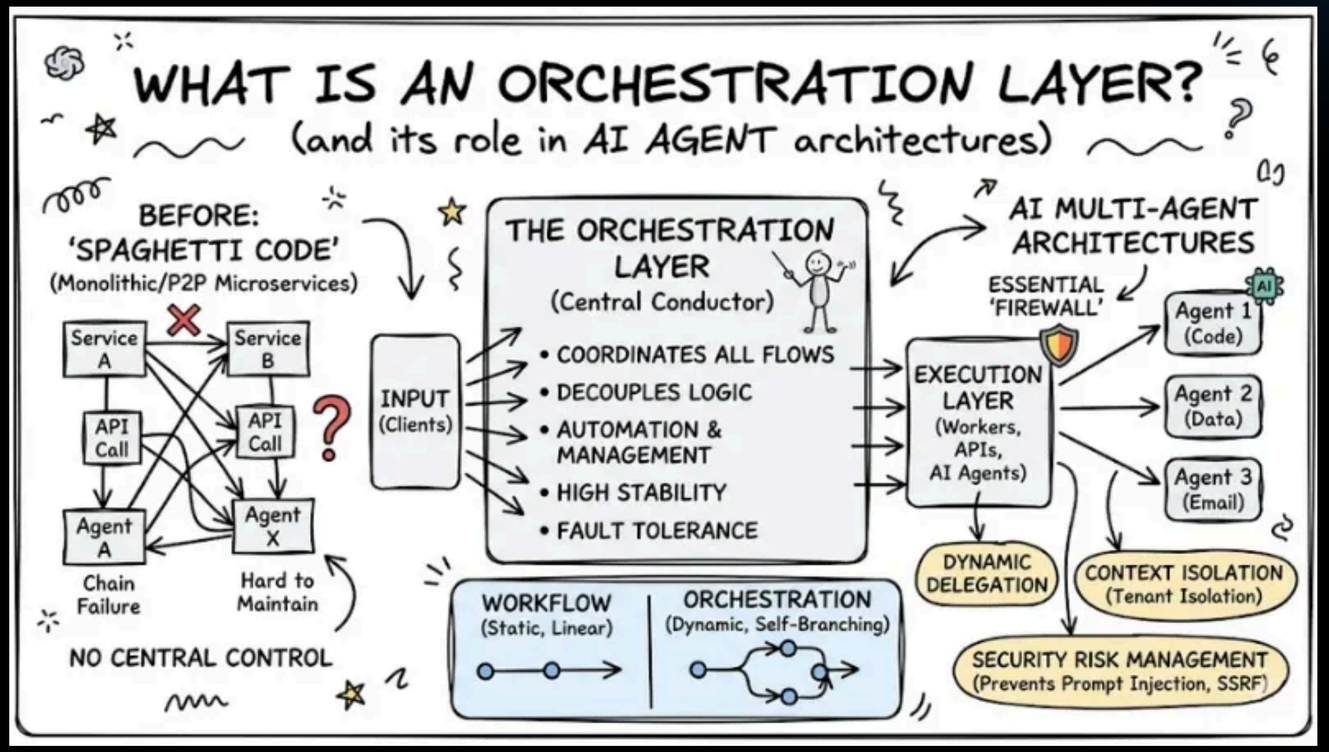
Shared components

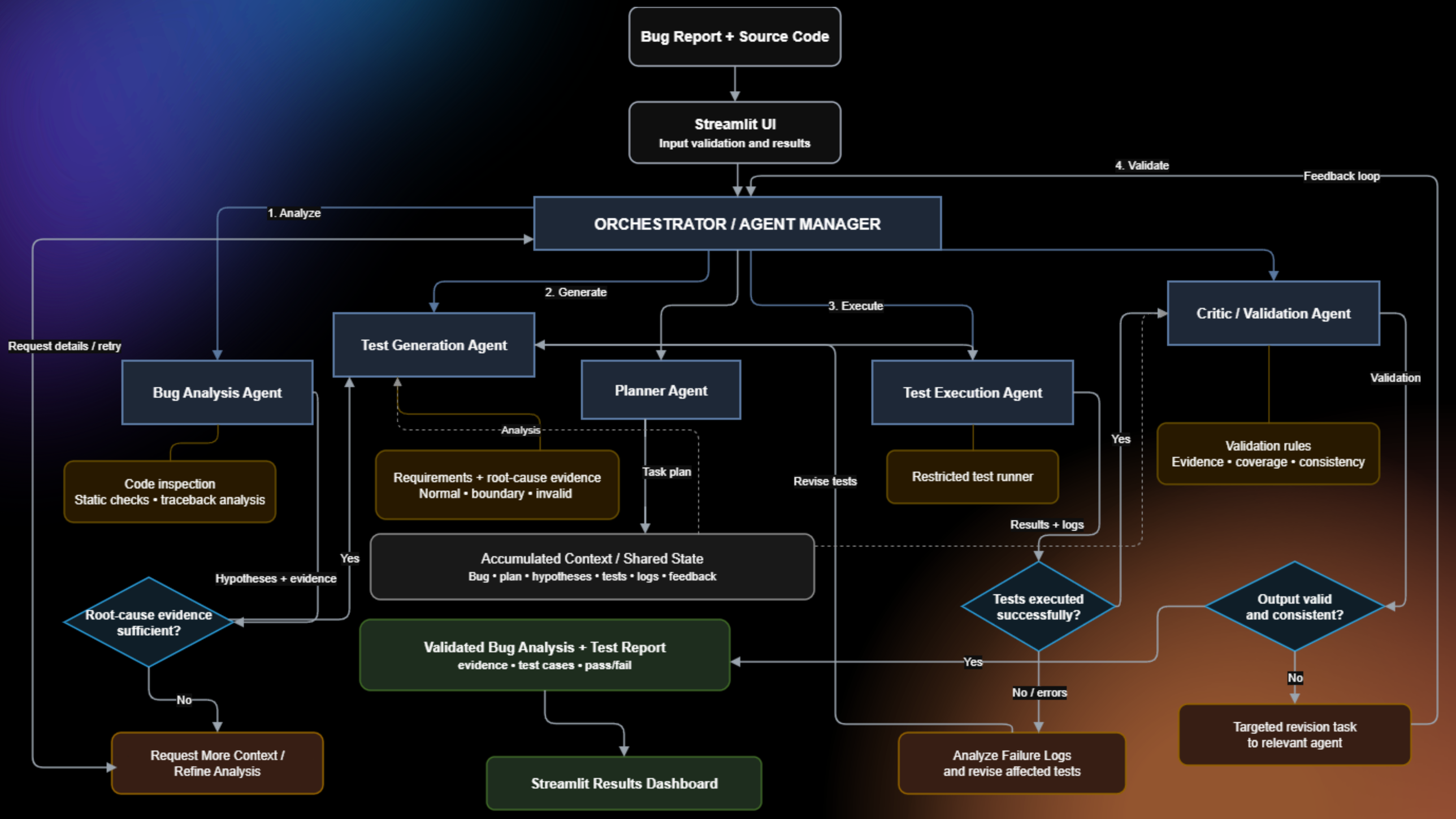
Orchestrator: Routes tasks, tracks state, manages dependencies, and decides what happens next.

Shared state: Stores the bug report, code, hypotheses, tests, execution logs, and validation feedback.

Tool layer: Python code analysis, test generation, and a controlled test runner.

Streamlit interface: Accepts user input and displays progress and final results.




Bug Report + Source Code
Streamlit UI
Input validation and results
4. Validate
Feedback loop
1. Analyze
ORCHESTRATOR / AGENT MANAGER
2. Generate
3. Execute
Request details / retry
Test Generation Agent
Critic / Validation Agent
Bug Analysis Agent
Planner Agent
Test Execution Agent
Validation
Analysis
Code inspection
Static checks • traceback analysis
Requirements + root-cause evidence
Normal • boundary • invalid
Task plan
Revise tests
Restricted test runner
Yes
Validation rules
Evidence • coverage • consistency
Results + logs
Yes
Hypotheses + evidence
Accumulated Context / Shared State
Bug • plan • hypotheses • tests • logs • feedback
Root-cause evidence sufficient?
Tests executed successfully?
Output valid and consistent?
Validated Bug Analysis + Test Report
evidence • test cases • pass/fail
Yes
No
No / errors
No
Request More Context / Refine Analysis
Streamlit Results Dashboard
Analyze Failure Logs and revise affected tests
Targeted revision task to relevant agent

How orchestration works

- The user submits a bug report and code through Streamlit.
- The orchestrator asks the Planner Agent to create a task plan.
- The Bug Analysis Agent investigates the code and identifies probable causes.
- The Test Generation Agent uses the analysis to create test cases.
- The Test Execution Agent runs the tests in a controlled environment.
- The Critic Agent checks the evidence and outputs. If something is incomplete or inconsistent, the orchestrator routes feedback to the relevant agent.
- The validated results are displayed in Streamlit.

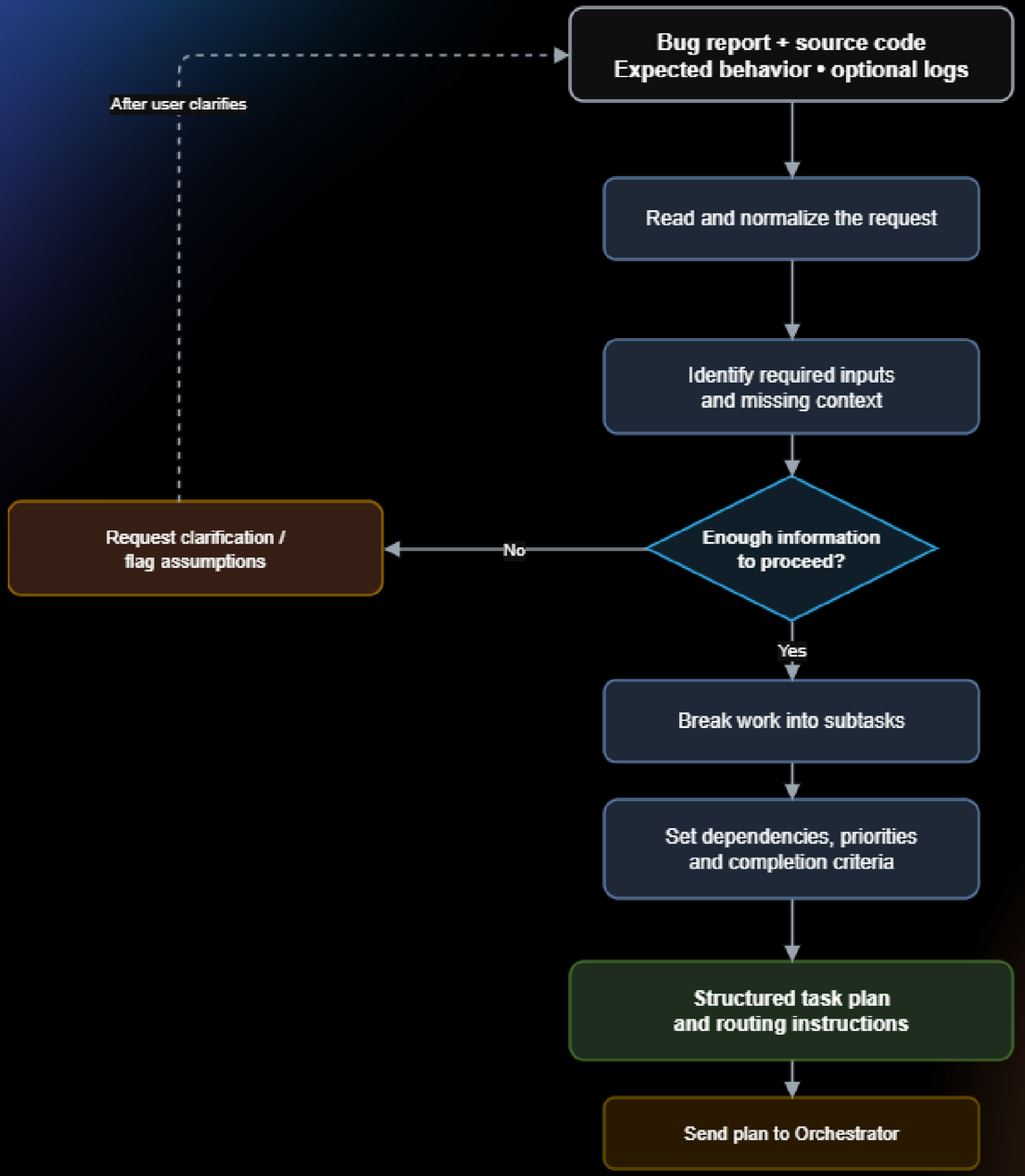
Detailed processing stages

- **Input validation:** Check that the report and code are provided and the selected language is supported.
- **Planning:** Determine what needs to be analyzed and tested.
- **Bug analysis:** Identify likely causes and the code paths involved.
- **Test design:** Translate each important hypothesis into testable scenarios.
- **Test execution:** Run the tests using a restricted test runner.
- **Validation:** Verify that the conclusions agree with the actual evidence.
- **Reporting:** Display the root-cause analysis, test cases, results, and unresolved limitations.

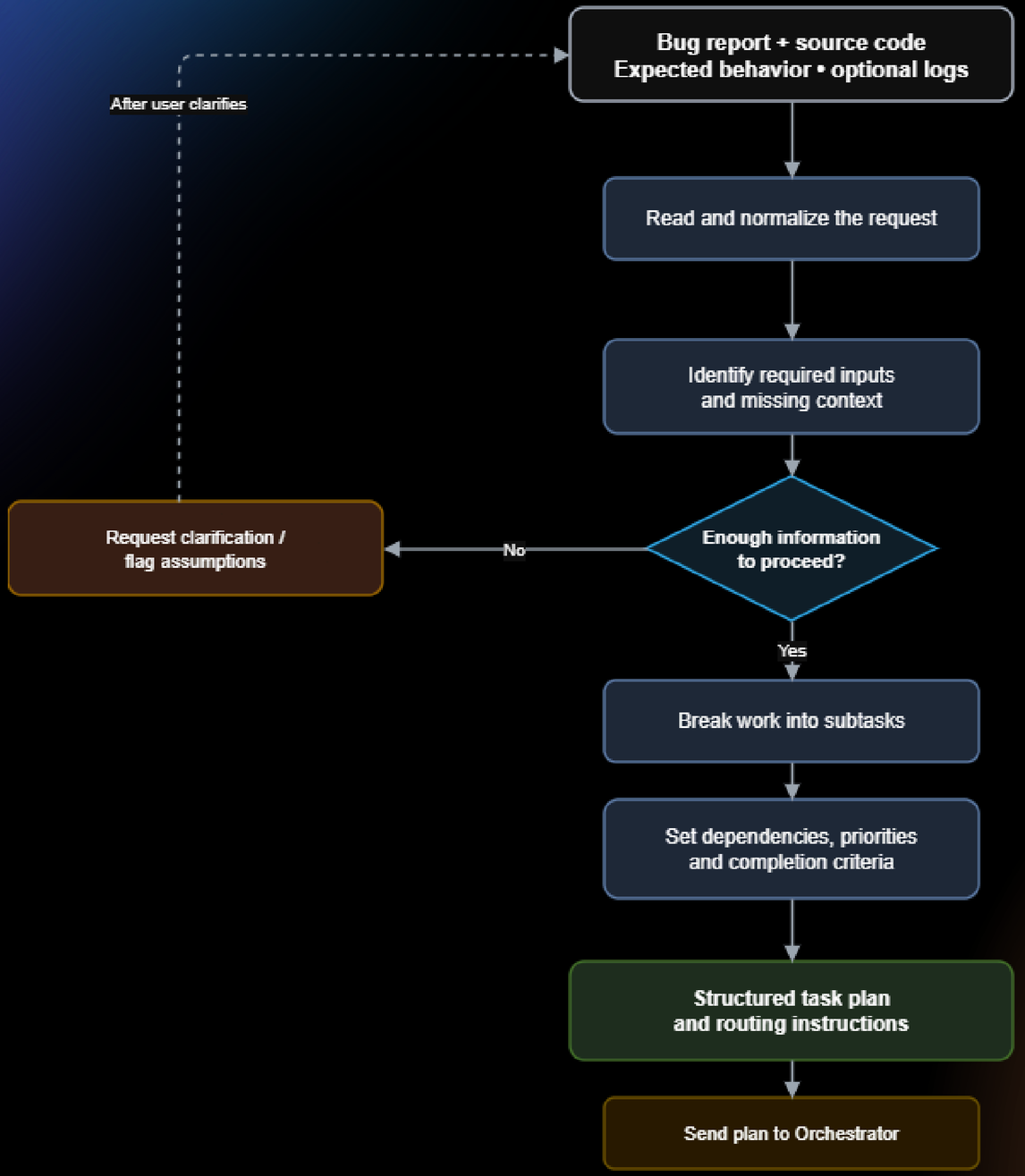
Decision-making rules

- **Insufficient evidence**: request more context or refine the analysis.
- **Invalid generated tests**: regenerate the affected tests.
- **Execution error**: inspect the logs and determine whether the test, environment, or application caused the failure.
- **Failed validation**: return targeted feedback to the relevant agent.
- **Retry limit reached**: report unresolved issues rather than claiming success.

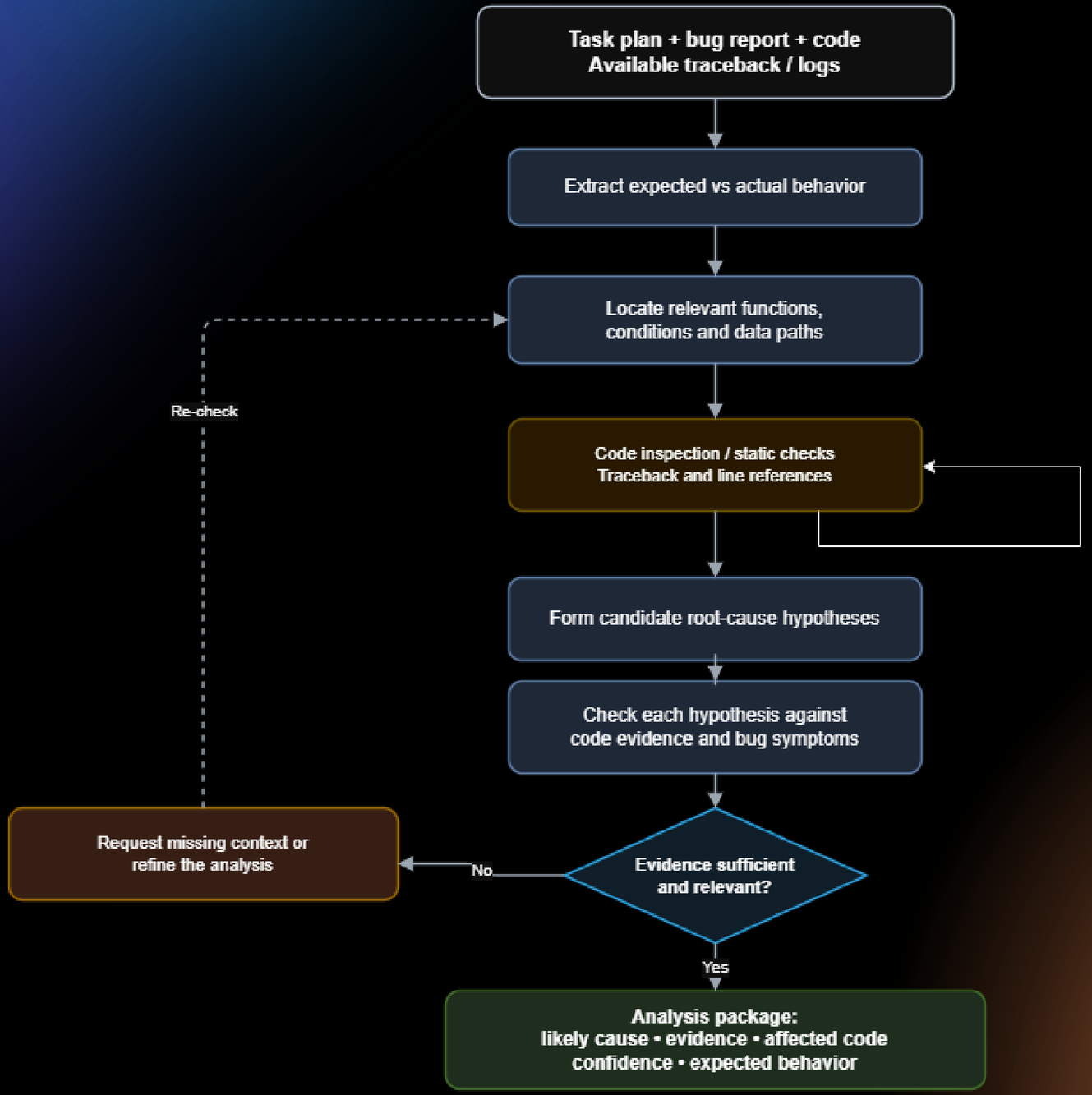
Planner Agent Workflow



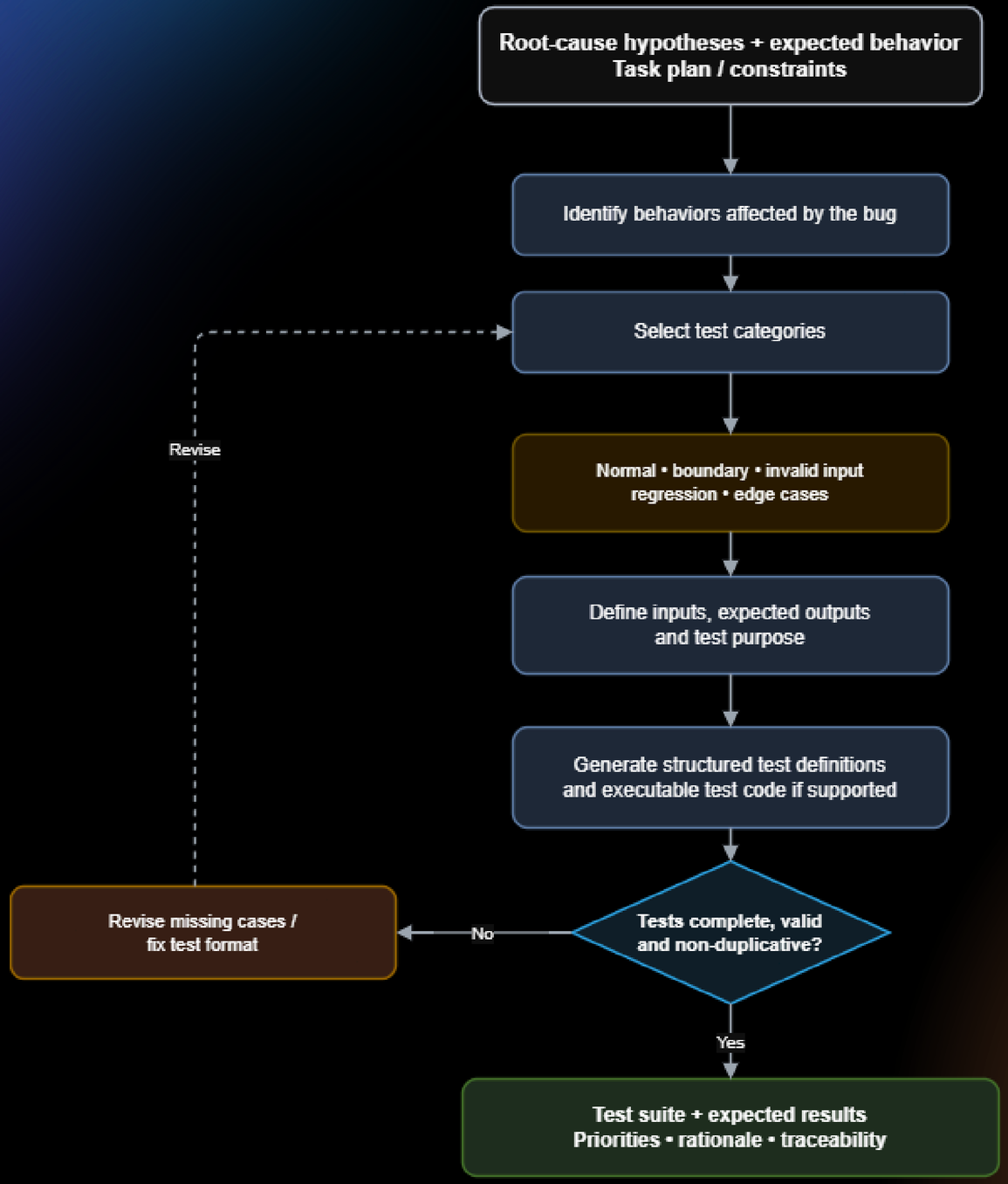
Planner Agent Workflow



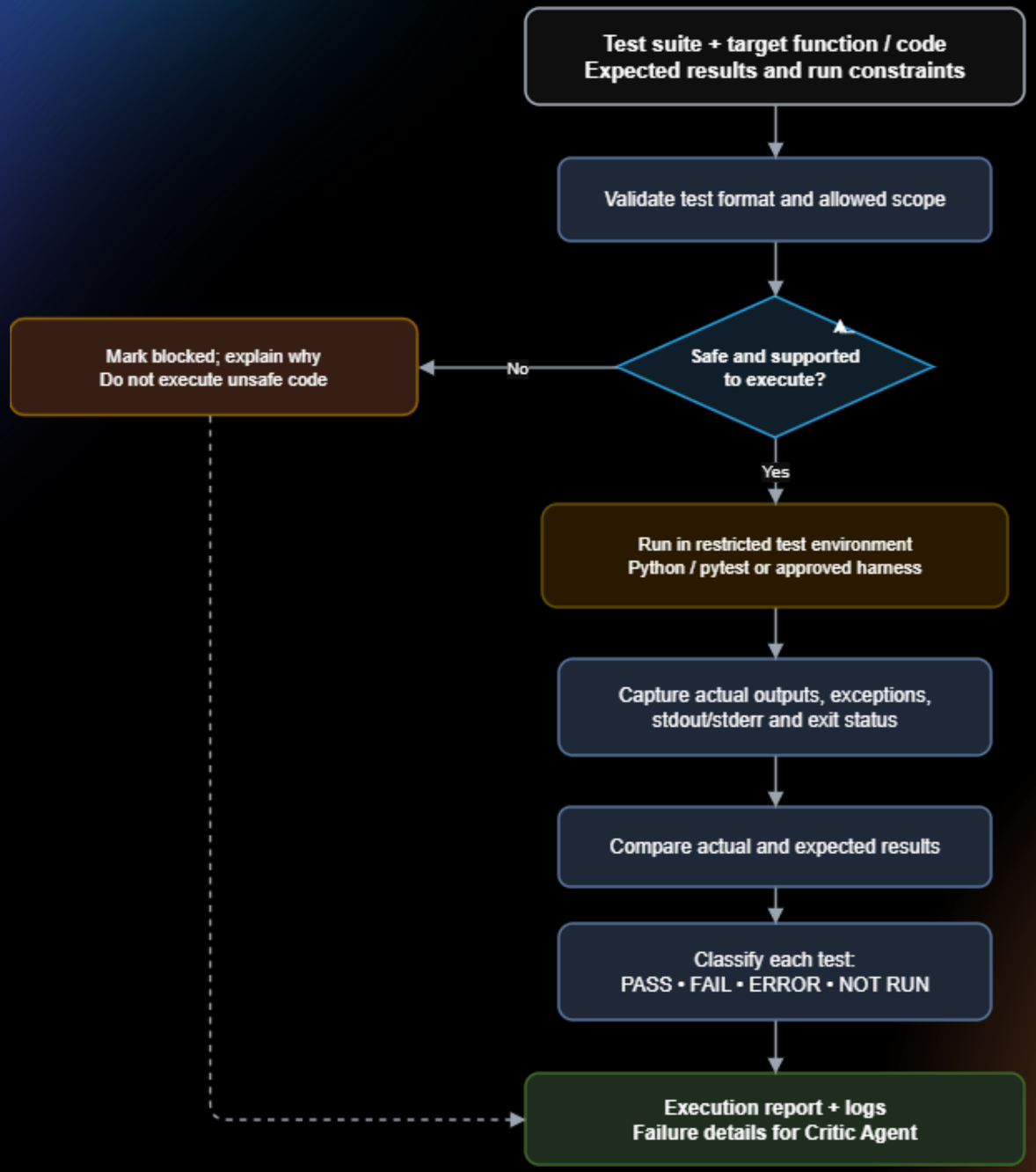
Bug Analysis Agent Workflow



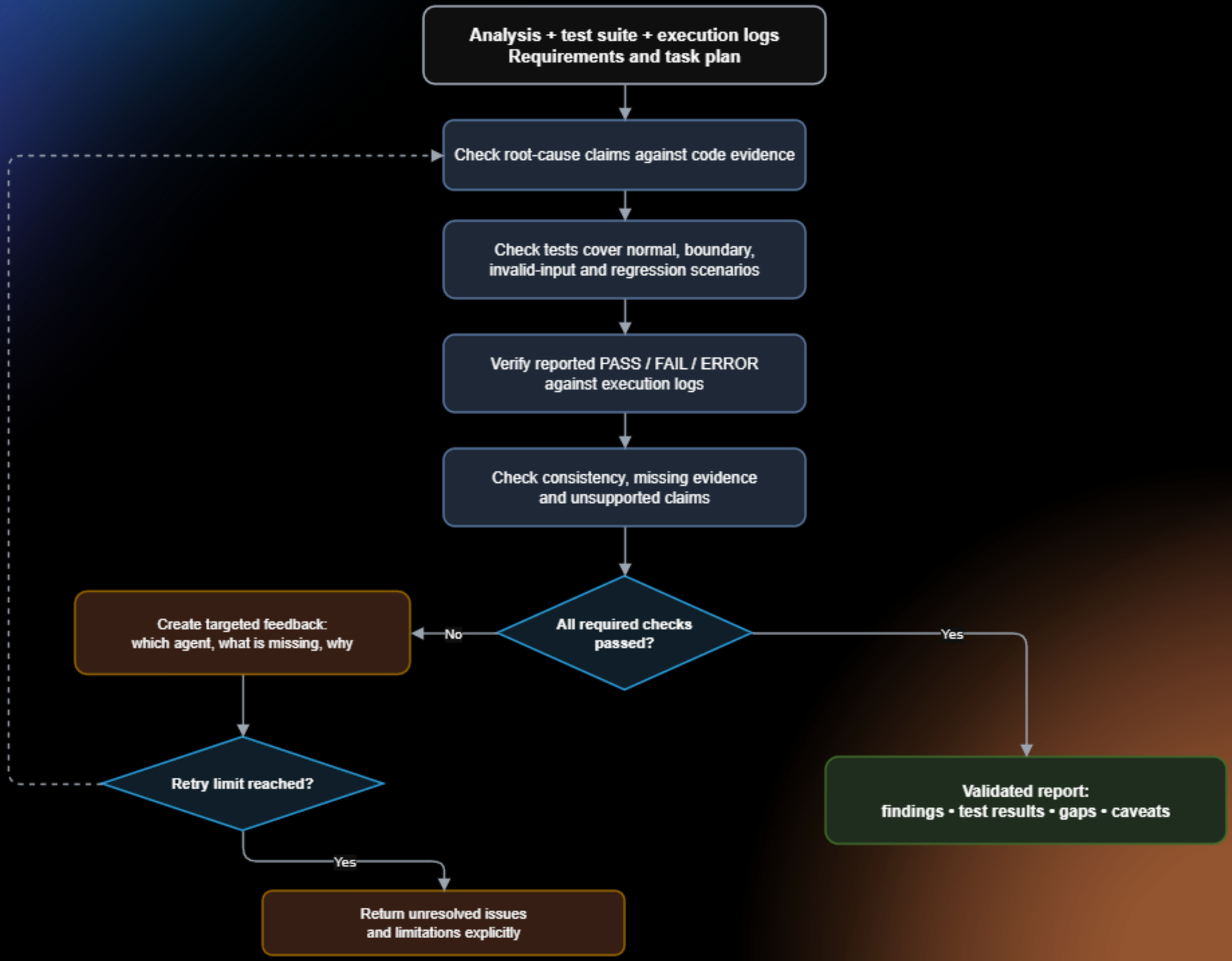
Test Generation Agent Workflow



Test Execution Agent Workflow



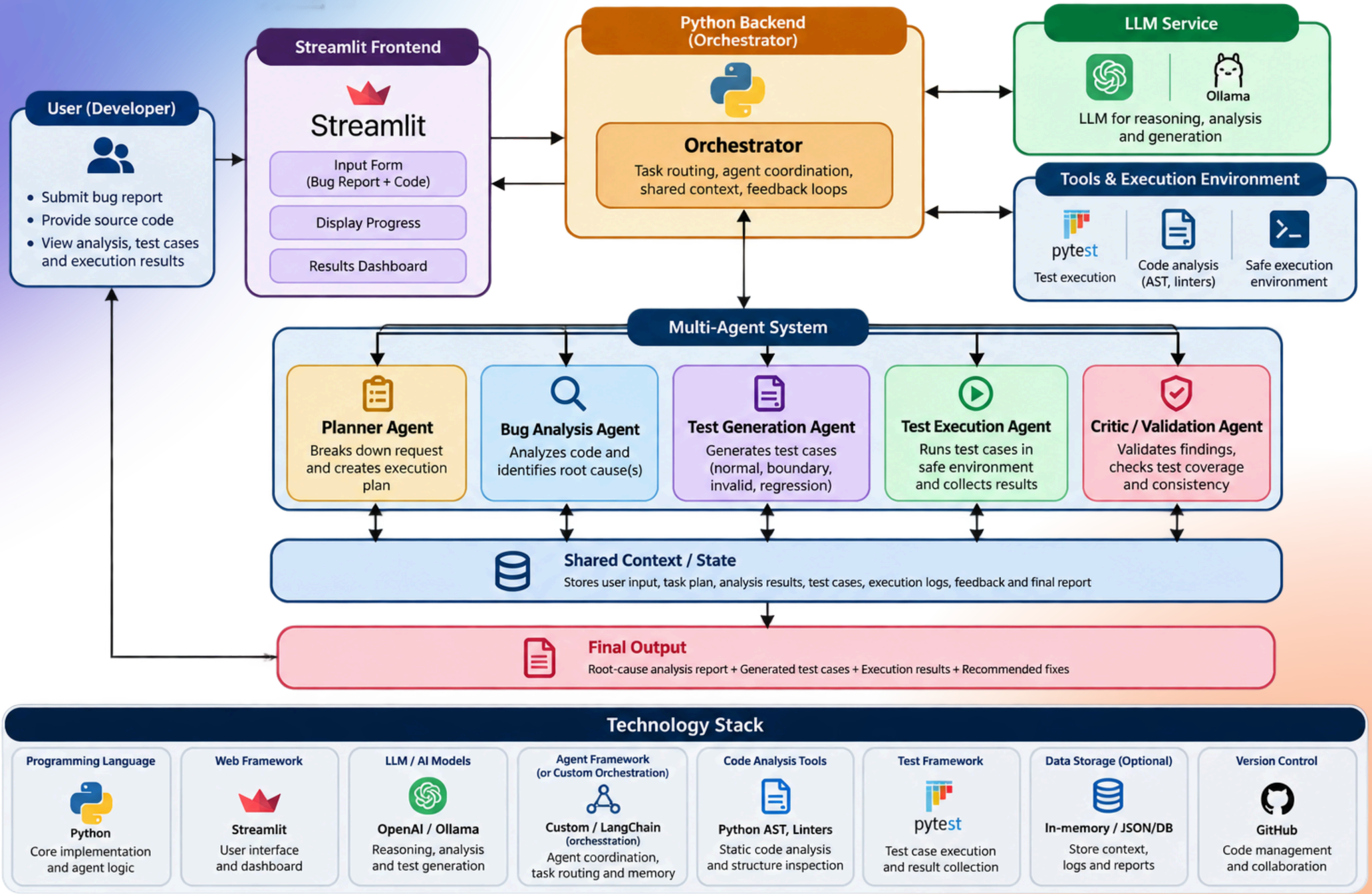
Critic / Validation Agent Workflow



Inter-agent handoffs

From	To	Information transferred
Planner	Bug Analysis	Analysis instructions and relevant inputs
Bug Analysis	Test Generation	Root-cause hypotheses and expected behavior
Test Generation	Test Execution	Test definitions and expected outcomes
Test Execution	Critic	Execution results and logs
Critic	Orchestrator	Approval or specific revision request
Orchestrator	Relevant agent	Corrective task and feedback

The agents should pass structured outputs rather than just free-form conversations. This makes the workflow easier to implement, test, and demonstrate.


User (Developer)
Submit bug report
Provide source code
View analysis, test cases and execution results
Streamlit Frontend
Streamlit
Input Form (Bug Report + Code)
Display Progress
Results Dashboard
Python Backend (Orchestrator)
Orchestrator
Task routing, agent coordination, shared context, feedback loops
LLM Service
Ollama
LLM for reasoning, analysis and generation
Tools & Execution Environment
pytest
Test execution
Code analysis (AST, linters)
Safe execution environment
Multi-Agent System
Planner Agent
Breaks down request and creates execution plan
Bug Analysis Agent
Analyzes code and identifies root cause(s)
Test Generation Agent
Generates test cases (normal, boundary, invalid, regression)
Test Execution Agent
Runs test cases in safe environment and collects results
Critic / Validation Agent
Validates findings, checks test coverage and consistency
Shared Context / State
Stores user input, task plan, analysis results, test cases, execution logs, feedback and final report
Final Output
Root-cause analysis report + Generated test cases + Execution results + Recommended fixes
Technology Stack
Programming Language
Python
Core implementation and agent logic
Web Framework
Streamlit
User interface and dashboard
LLM / AI Models
OpenAI / Ollama
Reasoning, analysis and test generation
Agent Framework (or Custom Orchestration)
Custom / LangChain (orchesstration)
Agent coordination, task routing and memory
Code Analysis Tools
Python AST, Linters
Static code analysis and structure inspection
Test Framework
pytest
Test case execution and result collection
Data Storage (Optional)
In-memory / JSON/DB
Store context, logs and reports
Version Control
GitHub
Code management and collaboration

Output Validation and Evaluation

Validation check	Method	Success condition
Root-cause evidence	Compare claims with source code	Explanation matches the
Test correctness	Check inputs, expected outputs,	Tests reflect stated requirements
Execution accuracy	Run against the actual function	Results match observed behavior
Coverage	Check normal, boundary, invalid,	Important scenarios are
Report consistency	Compare analysis with execution logs	Final claims agree with evidence

PoC success criteria

- Correctly identify the seeded bug in the demonstration.
- Generate the four expected discount test cases.
- Detect the two failing tests against the buggy implementation.
- Produce a corrected-code suggestion.
- Have the Critic Agent verify that the report agrees with the execution evidence.
- Display the complete workflow and final report in Streamlit.

Expected business impact

- Reduces repetitive manual analysis and test-writing effort.
- Helps developers identify likely root causes more systematically.
- Improves coverage of boundary and regression scenarios.
- Creates a structured, traceable debugging report.
- Supports faster feedback during software development.

Current limitations

- The PoC may struggle with large codebases and cross-file dependencies.
- Root-cause suggestions may be incorrect or incomplete.
- Mock agents demonstrate orchestration but do not provide full LLM-based reasoning.
- Generated tests are only as good as the supplied requirements and expected behavior.
- Test execution must be restricted to prevent unsafe code from running.

Future Scope

- Integrate repository tools such as GitHub for real project analysis.
- Add local LLMs through Ollama for natural-language reasoning.
- Extend testing to multiple programming languages and frameworks.
- Add code-diff analysis and automated regression testing.
- Track test coverage and evaluation metrics across repeated runs.

Conclusion

Multi-Agent Bug Analysis and Test Case Generation demonstrates how Agentic AI can transform software debugging from a largely manual process into a coordinated workflow.

By combining planning, bug analysis, test generation, execution, and independent validation, the system aims to produce more structured and evidence-based debugging results.

Key takeaway: The value of the system lies not in using multiple AI agents alone, but in how those agents collaborate, execute tools, evaluate results, and revise their work when necessary.